Desenvolvimento de um Serviço de Metadados Compatível com Cloud-Init para Incus

Gustavo Santos

Orientador: Prof. Maycon L. M. Peixoto





Agenda

1. Introdução
2. Trabalhos Relacionados
3. Proposta
4. Resultados
5. Conclusão

Introdução

Contexto

- Nuvens públicas (AWS, GCP, Azure) provisionam instâncias de forma programática e automatizada.
- Peça central: *Instance Metadata Service* (IMDS), acessível pelo endereço *link-local* `169.254.169.254` [**?**].
- Cada instância descobre a própria configuração no *boot*: hostname, endereços IP, chaves SSH, *user-data*.
- O **cloud-init** [**?**] consome esses dados e é o padrão da indústria para inicialização automatizada de instâncias.

Problema

- O **Incus** [**?**] (*fork* comunitário do LXD [**?**]) não oferece um serviço de metadados acessível via rede (TCP/IP).
- A entrega de configuração existente (*datasource* LXD via *socket* `/dev/incus/sock`) sofre uma **condição de corrida**: o *Incus agent* inicia tarde demais para os primeiros estágios do cloud-init em VMs [**?**].
- Consequências:
  - configuração manual ou *scripts* ad hoc;
  - sem portabilidade de configurações entre nuvem pública e privada;
  - OpenStack resolve, mas exige a plataforma completa, desproporcional para *homelabs*, borda e nuvens privadas pequenas.

Objetivo e Contribuições

Objetivo: projetar e implementar um serviço de metadados compatível com cloud-init para o Incus, leve e autônomo.

1. Análise das implementações de serviços de metadados em nuvens públicas e privadas e dos requisitos mínimos de compatibilidade com o cloud-init.
2. Projeto e implementação do serviço: API REST, sincronização de instâncias orientada a eventos, persistência e alta disponibilidade.
3. Avaliação em ambiente Incus real: corretude funcional, desempenho, escalabilidade e *failover*.

Trabalhos Relacionados

Serviços de Metadados Existentes

- **AWS IMDS** [**?**]: API HTTP em `169.254.169.254`; IMDSv2 adiciona autenticação por sessão contra SSRF.
- **GCP Metadata Server** [**?**]: exige o cabeçalho `Metadata-Flavor: Google`; escopos de projeto e de instância.
- **OpenStack Metadata** [**?**]: compatível com a API EC2, mas fortemente acoplado ao Neutron e ao Nova; não executa de forma autônoma.
- **Datasource NoCloud** [**?**]: obtém dados de uma URL base (`seedfrom`) consultando `/meta-data`, `/user-data` e `/vendor-data` (`network-config` só de *seed* local); exige apenas `instance-id` e `local-hostname`.
- **RAFT** [**?**]: consenso distribuído compreensível (eleição de líder + replicação de log); base da alta disponibilidade.

Comparação entre as Abordagens

Solução	Autônomo	Endpoint HTTP cloud-init	Contêineres	VMs	Alta Disp.	Cód. Aberto
AWS IMDS	✗	✓	✗	✓	✓	✗
GCP Metadata	✗	✓	✗	✓	✓	✗
OpenStack Metadata	✗	✓	✗	✓	✓	✓
LXD/Incus (atual)	✓	✗	✓	✓	✗	✓
Este trabalho	✓	✓	✓	○	✓	✓

○: suportado pelo projeto, mas não avaliado experimentalmente.

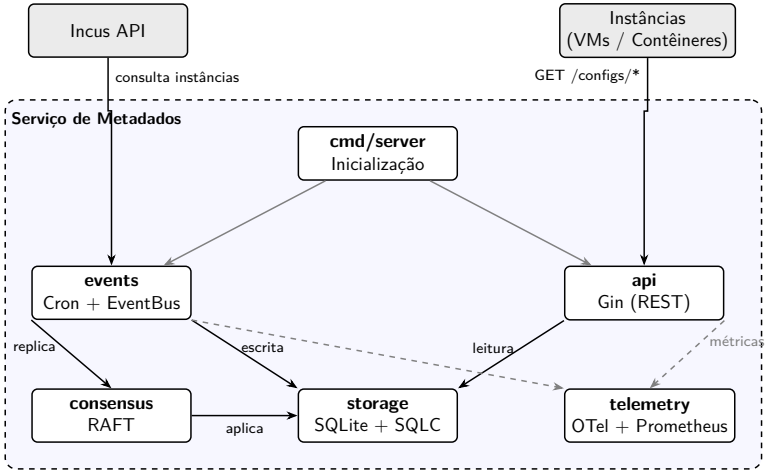
Lacuna: na revisão realizada, não foi identificado serviço de metadados leve e autônomo que implemente o protocolo HTTP esperado pelo cloud-init para o Incus.

Proposta

Visão Geral

- Serviço de metadados compatível com cloud-init para instâncias Incus (projetado para contêineres e VMs; avaliado com contêineres).
- API REST consultada pelas instâncias durante o *boot*, seguindo o modelo do *datasource* NoCloud HTTP.
- A instância é identificada pelo **endereço IP de origem** da requisição: sem agentes na instância; a única configuração é o `seedfrom` do NoCloud, definido uma vez na imagem-base.
- Sincronização orientada a eventos mantém o estado consistente com o gerenciador Incus.
- Implantação autônoma: um único binário Go, banco embutido, sem dependências externas.

Arquitetura



Sincronização de Instâncias

Ciclo periódico e orientado a eventos (*pub/sub*):

1. *Cron* a cada 10 s publica o evento `instances.sync`.
2. O *handler* consulta a API do Incus e lista as instâncias ativas.
3. Para cada instância, um evento `instance.sync` é publicado, com processamento desacoplado e paralelo.
4. Para cada instância, o serviço:
   - extrai rede (IPv4, IPv6, MAC) do estado reportado pelo Incus;
   - persiste metadados cloud-init (`instance-id`, hostname, chaves SSH, *placement*);
   - armazena `user-data` e `vendor-data` das chaves `cloud-init.*` do Incus;
   - gera `network-config` (Networking Config v2 / Netplan) quando não definida pelo administrador.

Endpoints da API

Endpoints públicos, no modelo do *datasource* NoCloud HTTP [**?**]:

- `GET /configs/meta-data`: identidade da instância (`instance-id`, hostname, IPs, `public-keys`).
- `GET /configs/meta-data/:key`: campo específico dos metadados.
- `GET /configs/user-data`: *scripts*/cloud-config do usuário.
- `GET /configs/vendor-data`: por instância, com *fallback* global.
- `GET /configs/network-config`: definida pelo administrador ou gerada automaticamente; não consultado pelo NoCloud via HTTP.

Endpoints internos (`/internal/vendor/*`): criação, consulta e atualização de vendor-data por fornecedor; não expostos às instâncias.

Descoberta do Serviço pelas Instâncias

- *Datasource* NoCloud em modo rede: `seedfrom` aponta para `/configs/` do serviço; o cloud-init concatena `meta-data`, `user-data` e `vendor-data`. O NoCloud não requisita `network-config` pelo `seedfrom`: a rede aplicada vem do *seed* local gravado pelo Incus.
- O `seedfrom` precisa existir antes de qualquer consulta: fica em um *drop-in* do cloud-init gravado na **imagem-base** (uma configuração por imagem, não por instância; sem agentes).
- Endereço do serviço, duas formas:
  - *link-local* `169.254.169.254` na *bridge* `incusbr0` + NAT da porta 80, para hosts dedicados;
  - *gateway* da *bridge* + porta do serviço, quando o host é uma VM de nuvem pública (endereço já em uso pelo provedor). Forma usada nos experimentos.

Alta Disponibilidade via RAFT

Nó único = ponto único de falha. Solução: consenso distribuído RAFT [**?**] com `hashicorp/raft` [**?**].

- **Eleição de líder**: apenas o líder sincroniza com o Incus e propõe escritas; falha do líder dispara nova eleição automática.
- **Replicação de log + FSM**: cada escrita vira um comando tipado, confirmado por maioria e aplicado ao SQLite local de cada nó.
- **Persistência**: log RAFT em BoltDB, sem banco externo.
- **Leituras em qualquer nó**: todos mantêm cópia local do estado (leituras eventualmente consistentes), distribuindo a carga de consultas.
- Ativação opcional por variáveis de ambiente (`RAFT_ENABLED`, `NODE_ID`, `PEERS`, ...).

Tecnologias Utilizadas

- **Go**: binário estático único; goroutines para o processamento paralelo de eventos.
- **Gin** [**?**]: framework HTTP de alto desempenho; separa rotas públicas e internas.
- **SQLite + SQLC** [**?**]: banco embutido; queries SQL com tipagem estática gerada.
- **GoEventBus**: barramento *pub/sub* que desacopla agendamento e sincronização.
- **gocron**: agendamento do ciclo de sincronização (10 s).
- **hashicorp/raft** [**?**]: consenso distribuído.
- **OpenTelemetry + Prometheus** [**? ?**]: métricas de sincronização e de API em `/metrics`.

Resultados

Metodologia de Avaliação

- Ambiente: GCE `e2-standard-4` (4 vCPUs, 15 GiB), Ubuntu 24.04 LTS, Incus 6.0.0; serviço no mesmo host das instâncias. Alta disponibilidade: 3 VMs `e2-standard-2`, uma réplica em cada.
- De 5 a 200 contêineres em execução, criados de uma imagem-base com o `seedfrom`; carga HTTP fixa com `hey` (5000 requisições, concorrência 50), percentis p50/p95/p99.
- SQLite em modo WAL, *busy timeout* de 5 s e uma única conexão (`SetMaxOpenConns(1)`).
- **Achado prático**: em nuvem pública, `169.254.169.254` já é usado pelo provedor; o *datasource* foi apontado para o *gateway* da *bridge* (`10.10.10.1:8080`).
- A imagem-base desabilita a rede do cloud-init (`eth0` por DHCP via Netplan). Experimento complementar com a rede habilitada: o NoCloud (cloud-init 26.1) não requisita `network-config` via HTTP.

Avaliação Funcional

Propriedade	**Resultado**
Isolamento de metadados	Aprovado
Isolamento sob manipulação do cabeçalho `X-Forwarded-For`	Aprovado
Execução de *user-data*	Aprovado
Entrega de *network-config*	Servido; não consultado via HTTP
Propagação de estado após reinício	Aprovado

Inicialização real: cloud-init selecionou o *datasource* NoCloud de rede, obteve *meta-data* e *user-data* (HTTP 200) e concluiu com `cloud-init status: done`: compatibilidade de ponta a ponta para esses dados.

Desempenho: Latência da API

Endpoint	p50 (ms)	p95 (ms)	p99 (ms)
`/meta-data`	13,2	53,7	83,2
`/user-data`	18,9	53,6	72,4
`/network-config`	11,4	47,2	72,4
`/vendor-data`	17,0	48,4	68,9

- Medianas de 11–19 ms; p99 abaixo de 90 ms, compatível com as janelas de tempo do *boot*.
- Latência de sincronização (criação no Incus $\rightarrow$ metadados disponíveis): $\approx$ 20,5 s em uma execução (ciclo de 10 s + *boot* e DHCP).

Desempenho: Escalabilidade

Instâncias	p50 (ms)	p95 (ms)	p99 (ms)	Erros	Mem. host (MB)
5	19,3	77,7	117,4	0,0%	1216
25	12,7	52,5	78,5	0,0%	2053
50	12,6	53,5	78,5	0,0%	3147
100	14,3	60,0	94,9	0,0%	4715
150	14,9	69,1	124,8	0,0%	4909
200	13,2	55,6	85,4	0,0%	5171

- Varia só o número de contêineres; a carga HTTP é a mesma em todas as linhas (5000 requisições, **concorrência fixa de 50**). Não mede muitas instâncias consultando ao mesmo tempo.
- Degradação por escrita serializada do SQLite **não corroborada** até 200 contêineres: latência sem tendência de alta e zero erros (leituras dominam; escritas a cada 10 s absorvidas pelo WAL). Uma execução por configuração.

Alta Disponibilidade (*cluster* de 3 nós)

Propriedade	**Resultado**
Formação do *cluster*	1 líder + 2 seguidores
Replicação de estado	instância presente nos 3 nós
Reeleição após falha do líder	**2,04 s** em média (10 execuções; 1,46 a 2,65 s)
Dados após *failover*	preservados
Reintegração de nó	reingressa como seguidor

- 2,04 s é o intervalo **sem líder**, não a indisponibilidade percebida pelas instâncias (não medida): seguidores seguem respondendo leituras; a sincronização volta no ciclo seguinte (até 10 s); instância apontada para o nó que caiu fica sem resposta (sem VIP/balanceador).
- Topologia: réplicas sobre um **Incus compartilhado**, que continua ponto único de falha para a sincronização (cenário não testado).

Discussão e Limitações

- Cobre os campos essenciais dos módulos mais comuns do cloud-init (`set_hostname`, `users_groups`, `write_files`, `runcmd`); rede apenas servida.
- **Limitações**:
  - SQLite serializa escritas: potencial gargalo bem acima da escala avaliada;
  - subconjunto dos campos de AWS/GCP (sem tipo de instância, faturamento);
  - *failover* pressupõe o Incus compartilhado, que segue como ponto único de falha;
  - correções posteriores na geração do `network-config` não foram reavaliadas experimentalmente.
- **Ameaças à validade**: host único nos testes de desempenho; contêineres de sistema (não VMs completas); janela de inconsistência do *polling*; `network-config` servido, mas não consumido pelo NoCloud via HTTP; sincronização medida em execução única; reeleição em 10 repetições.

Conclusão

Conclusão

- Serviço de metadados compatível com cloud-init para o Incus: preenche a lacuna de um endpoint HTTP autônomo em infraestrutura privada.
- Identificação por IP de origem: sem agentes nas instâncias; a única configuração é o `seedfrom` na imagem-base.
- Validado em ambiente real (contêineres de sistema, host único): *boots* completos com sucesso, latências de dezenas de milissegundos, latência estável até 200 contêineres e reeleição de líder em 2,04 s em média (10 execuções; intervalo sem líder).
- Código aberto, binário único, implantação simples, adequado a *homelabs*, borda e nuvens privadas de pequena escala.

Trabalhos Futuros

1. Eliminar a imagem-base pré-configurada: *datasource* nativo do Incus no cloud-init ou `seedfrom` via linha de comando do *kernel* (VMs).
2. Ampliação dos módulos e campos do cloud-init suportados.
3. Autenticação baseada em sessões inspirada no IMDSv2 [**?**] contra SSRF.
4. Substituição do *polling* pela API de eventos do Incus (sincronização em tempo real).
5. *Benchmarks* em maior escala, com repetições para análise estatística.
6. Levar o `network-config` a um caminho consumido pelo cloud-init (*datasource* via rede ou *seed* local do Incus); o conteúdo gerado foi corrigido após os experimentos, mas não reavaliado.

Obrigado!

Perguntas?

gustavoafs@ufba.br

Referências